CampusOS

Campus life, in sync.

Technical Design Document · Offline-first academic operations workspace
Prototype for one campus · B.Tech CSE · Faculty and Office Admin
Submitted for the Isocode Labs Software Engineering Intern take-home · November 2026

Source repository (public, verified): https://github.com/Sathvik2005/campusos
Live application: not deployed (see section 17 and `docs/stage-11/DEPLOYMENT_STATUS.md`).

> **Honesty statement.** Everything in this document marked as tested was run in the development environment (SQLite, mocked Supabase tokens, jsdom/IndexedDB emulation, and a real browser for the signed-out pages). Not verified: any signed-in browser session, PostgreSQL/Supabase, a live AI model call, and any production deployment. Sections 18 and 19 list these limits.

1. Executive summary

CampusOS lets a campus keep running when the internet does not. Faculty take attendance on a phone, even offline; the data waits safely on the device and syncs without duplicates. The Office Admin manages academic masters, internal marks (with moderation and locking), simulated fee ledgers, and an evidence-based view of students who may need attention. A deterministic rules engine decides risk; an optional AI layer may only explain the rule evidence and fails closed.

2. Business problem

Campus operations (attendance, marks, fees) depend on connectivity that classrooms often lack. Paper registers get re-keyed, marks change without a trail, and early warning for struggling students arrives too late. The product must therefore (a) never lose attendance, (b) make every sensitive change auditable, and (c) separate decisions (rules) from explanations (AI).

3. Proposed solution and MVP scope

A web application (Next.js) talking to a single authoritative API (FastAPI) backed by PostgreSQL on Supabase. Attendance is written to IndexedDB first, queued, and replayed idempotently.

In scope (built)	Out of scope / deferred

One campus, one programme (B.Tech CSE); two roles; students, academic masters, attendance (online + offline), internal marks lifecycle, simulated fees, risk insights + mentor tasks, audit history	Real payment gateway, multiple campuses/programmes, parent/student portals, Service-Worker background sync, student create/edit UI, rate limiting, CSP

4. User roles

Role	Can	Cannot
Faculty	See own assigned classes and students; take attendance (online/offline); enter and submit marks for assigned classes	See fees, other faculty's classes, Academics, Insights, Audit; correct attendance after sync
Office Admin	Manage masters and assignments; moderate/lock/unlock marks; correct attendance with a reason; fees and simulated payments; insights and mentor tasks; read the audit log	Edit or delete audit events; take attendance as faculty

Enforcement is server-side on every route and verified by an automated matrix over the route table (section 15).

5. Requirements (summary)

- **Functional:** authenticated access; role-scoped data; offline attendance with idempotent sync and conflict reporting; marks validation and lifecycle (draft, submitted, moderated, locked); fee ledger with partial payments; deterministic risk categories with stored evidence; mentor tasks with human approval; audit of important actions.
- **Non-functional:** no data loss on failure; responsive 360-1440 px; 44 px touch targets; no secrets in Git; documented limits.
- **Sample record that must match:** Subhashree Mohanty · ENG-24-CSE-0412 · B.Tech CSE · Year 2 · attendance 81.5% · internals 33/40 · fee due ₹0 (seeded and asserted in tests).

6. System architecture

```
Browser (Next.js App Router, TypeScript, Tailwind, TanStack Query, Dexie/IndexedDB)
   |  HTTPS + Authorization: Bearer <Supabase access token>
   v
FastAPI (Python, Pydantic v2, SQLAlchemy 2 sync, Alembic)   <- the only writer
   |  SQL
   v
PostgreSQL on Supabase          Supabase Auth issues JWTs; FastAPI verifies them via JWKS
```

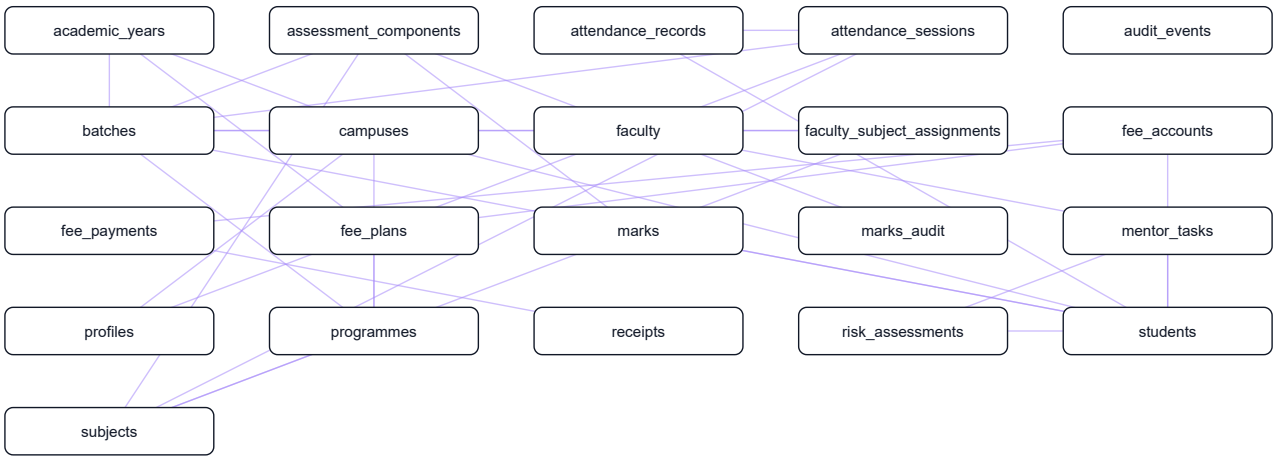
Modular monolith: no microservices, queues or Supabase client mutations. The browser never talks to Postgres and never holds a service-role key. Hosting target: Vercel (frontend), Render (backend), Supabase (database + auth).

7. Technology stack

Layer	Choice
Frontend	Next.js 16, React 19, TypeScript, Tailwind 4, TanStack Query, Dexie
Backend	FastAPI, Pydantic v2, SQLAlchemy 2 (sync), Alembic, Python 3.11 locally (3.12 targeted)
Data / auth	PostgreSQL (Supabase), Supabase Auth (JWT, JWKS)
Tests	Pytest + HTTPX, Vitest + Testing Library + fake-indexeddb, Playwright (Edge)

8. Database schema and ER diagram

21 tables, 34 foreign keys, one linear Alembic revision (`0001`). The diagram below is generated from the SQLAlchemy metadata (lines are foreign keys).



Integrity rules that matter most

- Attendance session: unique `idempotency_key`; unique `(batch, subject, held_on, period)`; period 1-8.

- Attendance record: unique `(session, student)`; status in `present|absent|late`.
- Payments carry a unique idempotency key; marks carry a lifecycle and an audit trail; audit events have no update/delete route.

▼ Table to referenced tables

Table	References
academic_years	campuses
assessment_components	batches, subjects
attendance_records	attendance_sessions, students
attendance_sessions	batches, faculty, subjects
audit_events	-
batches	academic_years, campuses, programmes
campuses	-
faculty	campuses
faculty_subject_assignments	batches, faculty, subjects
fee_accounts	fee_plans, students
fee_payments	fee_accounts
fee_plans	academic_years, programmes
marks	assessment_components, students
marks_audit	assessment_components
mentor_tasks	faculty, risk_assessments, students
profiles	campuses, faculty
programmes	campuses
receipts	fee_payments
risk_assessments	students
students	batches, campuses
subjects	programmes

9. API architecture

REST under `/api/v1` with a uniform error envelope `{"error": {"code","message","details"}}`. Public: `/health`, `/ready`. Everything else requires a Bearer token. Main groups: `me`; masters (`programmes, academic-years, batches, subjects, faculty, assignments`); `students`; `attendance` (`roster, sessions, sync, correction`); `assessments` + `moderation`; `accounts, payments, receipts`; `insights, mentor-tasks`; `audit`; `overview`.

10. Authentication and authorisation

1. The browser signs in with Supabase and receives a JWT.
2. Every API call carries it; FastAPI verifies signature (JWKS), audience and expiry, then loads the user's profile (role, campus, faculty id).
3. Dependencies `require_admin` / `require_faculty` guard routes; faculty queries are additionally scoped by teaching assignments; out-of-scope ids answer 404 (no existence signal).

The frontend guard only improves UX; the API is the enforcement point.

11. Attendance architecture and offline synchronisation

1. **Write IndexedDB first:** Save stores the complete session (the exact `/attendance/sync` body) in a Dexie `outbox` in one transaction, tagged with the profile id.
2. **Client UUID idempotency key** = outbox id. Replays send the same key; the server returns the existing session (200) instead of creating a duplicate.
3. **Sync runner** (single-flight) triggers on load, `online`, tab visible, a 30 s tick and manual retry; exponential backoff 2 s to 5 min.
4. **Server checks:** JWT, assignment, students in batch, complete roster, status values; unique constraints as the final guard.

Answer	Outcome (local data always kept)
200/201	synced
network error	stays pending, queue pauses
401	stays pending; "Sign in again to sync"
5xx/408/429	failed, retried with backoff or manually
403/404/422	failed, not retryable
409	conflict shown for review; never silently merged

The header chip reads `navigator.onLine` and the live outbox counts: conflict, failed, syncing, waiting, offline, synced. Rosters and class lists are cached per profile for offline marking; a roster that was never cached produces an honest message, not an empty list. No tokens or fee data are stored locally. Logout keeps pending records; another profile cannot send them.

> Limit: there is no Service Worker, so pending records sync the next time the app is opened online. The full browser journey (reload and close/reopen while offline, then reconnect) was tested with fake-indexeddb, not yet in a real signed-in browser.

12. Internal marks and moderation

Lifecycle: draft → submitted (faculty, all students scored) → moderated (admin) → locked (admin). Scores are validated (number, not negative, not above max, 2 decimals). Faculty edit only drafts; admins may edit submitted/moderated marks with a mandatory reason; locked marks are read-only until an admin unlocks them with a reason. Every score and state change writes a history row shown in the UI.

13. Fee management

Exact decimal arithmetic. Partial payments allowed, overpayment rejected (client and server). Each payment has an idempotency key; a replay returns the same receipt. Every amount is labelled simulated; no payment gateway exists. Reconciliation totals shown on the Fees list are per page.

14. AI student intelligence

Input	Rule (version rules-v1)	Label
Attendance	< 65% severe; 65% to < 75% shortage (75% is the stated line)	High / Moderate
Internals	< 40% of max	High
No data	nothing recorded	Insufficient data

The 65% and 40% thresholds are our documented choices, not institutional policy. The AI layer receives rule output only (no names or ids), must return validated JSON, may not contain numbers absent from the evidence, and on any failure the screen shows the deterministic evidence plus an "AI explanation unavailable" note. It never writes marks, attendance, fees or tasks. Mentor tasks require an explicit "Approve and create task". No AI key is configured, so no live model output exists.

15. Testing, QA and security evidence

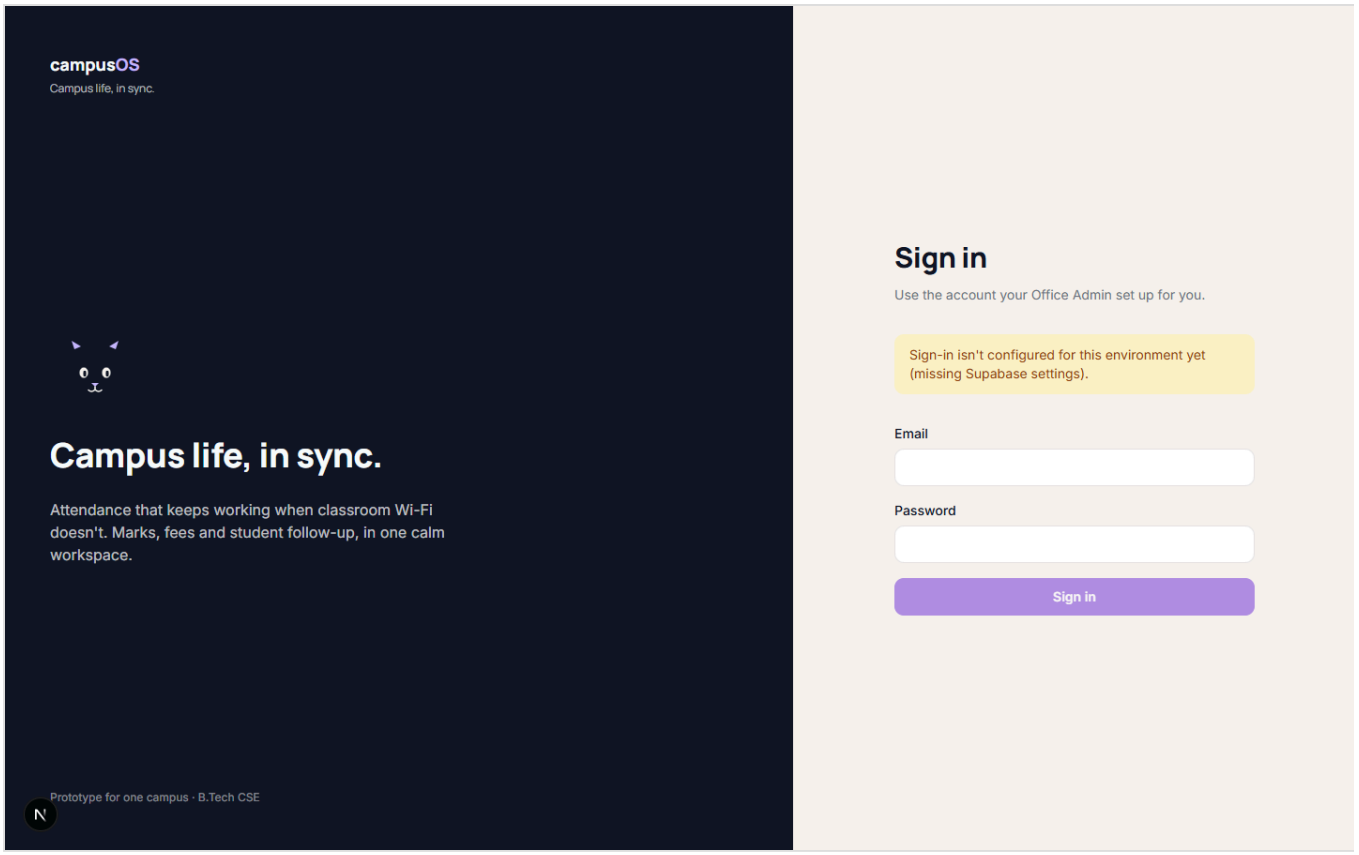
Check	Result
Backend (pytest)	151 passed (SQLite)
Frontend (Vitest)	89 passed
Browser (Playwright, Edge)	12 passed (sign-in page at six widths, redirects, real CORS to FastAPI, security headers)
Typecheck / lint / production build	clean / clean / pass
Dependency audit	npm audit (prod): 0; pip-audit: 0
Route security matrix	every API route rejects anonymous callers; admin-only routes reject faculty; faculty-only reject admin
Migration drift	alembic check: none (SQLite)

Security controls: server-side authorisation on all routes; 404 for out-of-scope ids; Pydantic request schemas (no mass assignment); parameterised SQL; explicit CORS allow-list without credentials; production docs disabled; baseline security headers; no secrets in Git (scan clean); audit log is read-only. Open items: rate limiting, CSP, real JWKS/Postgres verification.

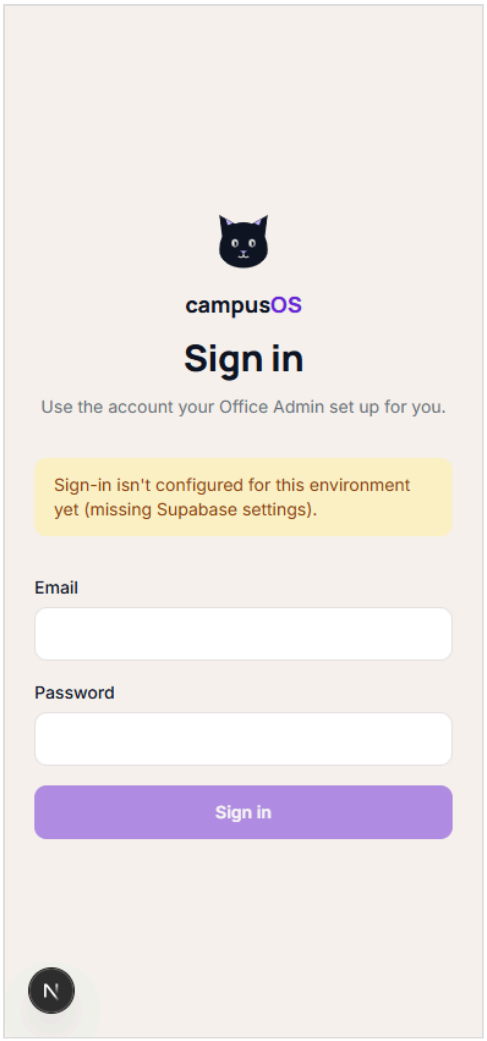
16. UI, responsive design and implementation screenshots

One shell with seven templates (Overview, Students, Academics, Attendance, Marks, Fees, Insights) plus Audit; sidebar on desktop, bottom navigation for faculty on small screens; the sync chip is in the header at every width. Palette Midnight/Paper/Lilac/Mint; Manrope headings, Inter body.

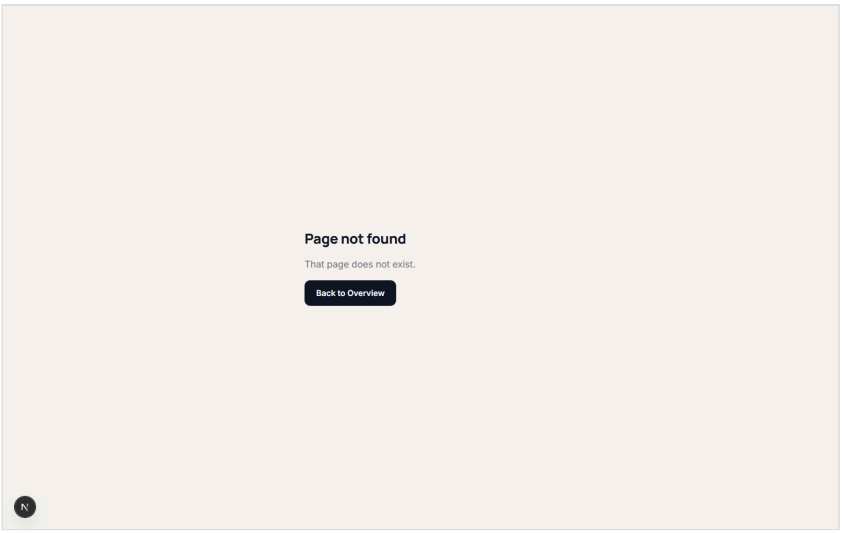
Screenshots below are real captures from the running app (development server). Signed-in screens could not be captured because no Supabase project was available; this is a limitation of the evidence, not of the design.



Sign-in, 1440 px (honest 'not configured' notice because no Supabase settings exist locally)



Sign-in, 390 px



Not-found page, 1440 px

17. Deployment architecture and status

Target: Vercel (root `frontend`) + Render (`render.yaml` blueprint, root `backend`) + Supabase. Migration policy: reviewed, backed up, applied manually with owner approval; never on app start. **Status: prepared but not deployed** (no Supabase project or deployment credentials; no URLs). Details: `docs/stage-11/`.

18. Coverage against the client brief

The brief asks for a thin, end-to-end slice (one campus, one programme, faculty and office admin, about 6 to 8 screens) with offline attendance as the hard part. Everything else in "Must have" is optional depth.

Brief item	Status
Masters for one programme (years, classes, subjects, faculty, assignments)	Built (Academics)
Mobile attendance, period-wise, working offline in class, sync, shortage at 75%	Built; offline queue verified in jsdom/IndexedDB emulation, real signed-in browser run pending. 75% drives the risk rules and the student view; automatic alert delivery is not built

Internal marks by component, moderation lock, every change logged	Built (component, draft/submit/moderate/lock/unlock, history). Mark sheets and university upload-format export are not built
Fee dues, receipts, mock payments	Built (simulated ledger, partial payments, receipts). Fee structures by category, scholarships, payment link and Tally export are not built
AI at-risk flag that only creates a mentor task	Built: rules decide, optional AI explains, task needs human approval
Plain-English read-only query assistant (optional depth)	Not built
Seed from the sample data	The B.Tech CSE sample row (Subhashree Mohanty, ENG-24-CSE-0412, 81.5%, 33/40, fee due 0) is seeded and tested. The Mechanical and MBA rows belong to other programmes/campuses and were left out on purpose to keep the slice to one programme
Admissions, staff leave, multi-campus dashboards, reports for university/AICTE/NAAC, Odia messages, parent view	Optional depth; not built

Constraints. DPDP 2023: the demo uses synthetic data only (plus the one specified sample record); parental consent for minors, data-principal rights and retention are not modelled and would be required before real use. Campus/department scoping is enforced for faculty through teaching assignments; per-campus roles for several campuses are not built. Marks lock after moderation with a full change log. Results-day load (7,800 students at once) was not load tested. Weak classroom Wi-Fi is the reason for the offline-first design.

19. Limitations

- No signed-in browser verification, no PostgreSQL run, no live AI call, no Lighthouse/axe/screen-reader pass.
- No Service Worker background sync; no rate limiting; no CSP.
- Academic model has no separate semester/department entities (one programme; a class = year + section).
- Mentor task form lacks assignee and completion-note fields; no student create/edit UI; no dashboard trend cards; fee reconciliation is per page.

20. Future improvements

Service-Worker background sync; push of conflict resolution to the admin; rate limiting and CSP; Postgres-backed CI with Playwright journeys for both roles; semester/department entities; mentor assignment and notes; campus-wide fee reconciliation; multi-campus support.

21. References

- Repository docs: `docs/phase-0` to `phase-12`, `docs/stage-6` to `stage-12` (reports and evidence)

- Offline design: `docs/phase-2/OFFLINE_SYNC_DESIGN.md`; report: `docs/stage-8/OFFLINE_ATTENDANCE_REPORT.md`
- Security: `docs/stage-10/SECURITY_AUDIT.md`; QA: `docs/stage-10/QA_MASTER_REPORT.md`